

Project Name: Project 2: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/10**Test Case ID#:** 1**Name(s) of Testers:** Mark Tsai**Test Description:**

Ballots class constructor with no shuffle test

The test is in testBallots.cpp with the test name

ConstructorNoShuffleTest. The Ballots constructor is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.**Automated:** yes**Results:** Pass**Preconditions for Test:**

Assume the ballot files do not have data errors based on the project description, no exception handling for the ballots needed.

A candidate vector contains the candidate names, notShuffleCandidatesVector, is created, and a ballot vector that contains the corresponding ballot information, notShuffleBallotsVector, is created. These two vectors are used to create a Ballots class object, notShuffleBallots, with the Ballots class constructor and shuffle option “false”.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Retrieve all ballots within the Ballots object, notShuffleBallots, and check whether the ballots are shuffled or not.	notShuffleBallots notShuffleBallotsVector	The ballots retrieved should not be shuffled compared to notShuffleBallotsVector.	The ballots retrieved are in the same order as notShuffleBallotsVector.	None

Post condition(s) for Test:

The ballots stored in the Ballots class object, notShuffleBallots, aren't shuffled. Therefore, they match notShuffleBallotsVector. System state remains unchanged.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/10**Test Case ID#:** 2**Name(s) of Testers:** Mark Tsai**Test Description:**

Ballots class constructor with shuffle test

The test is in testBallots.cpp with the test name

ConstructorShuffleTest. The Ballots constructor is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.**Automated:** yes**Results:** Pass**Preconditions for Test:**

Assume the ballot files do not have data errors based on the project description, no exception handling for the ballots needed.

A candidate vector contains the candidate names, shuffleCandidatesVector, is created, and a ballot vector that contains the corresponding ballot information, shuffleBallotsVector, is created. These two vectors are used to create a Ballots class object, shuffleBallots, with the Ballots class constructor and shuffle option "true".

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Retrieve all ballots within the Ballots object, shuffleBallots, and check whether the ballots are shuffled or not.	shuffleBallots shuffleBallotsVector	The ballots retrieved should be shuffled and therefore different from shuffleBallotsVector.	The ballots retrieved have a different order compared to shuffleBallotsVector.	None

Post condition(s) for Test:

The ballots stored in the Ballots class object, shuffleBallots, are shuffled. Therefore, they don't match shuffleBallotsVector. System state remains unchanged.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/29**Test Case ID#:** 3**Name(s) of Testers:** Mark Tsai**Test Description:**

Ballots class getBallot function test.

The test is in testBallots.cpp with the test name GetBallotTest.

The Ballots getBallot function is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.**Automated:** yes**Results:** Pass**Preconditions for Test:**

Assume the ballot files do not have data errors based on the project description, no exception handling for the ballots needed.

A candidate vector contains the candidate names, notShuffleCandidatesVector, is created, and a ballot vector that contains the corresponding ballot information, notShuffleBallotsVector, is created. These two vectors are used to create a Ballots class object, notShuffleBallots, with the Ballots class constructor and shuffle option "false".

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Retrieve each ballot in the Ballots object by index, notShuffleBallots, and compared with the ballot in notShuffleBallotsVector with the same index.	notShuffleBallots notShuffleBallotsVector	Each ballot by index in notShuffleBallots should be the same as the ballot by index in notShuffleBallotsVector.	The ballots retrieved by index in notShuffleBallots are the same as the ballots retrieved by index in notShuffleBallotsVector.	None
2	Test the out-of-range index when retrieving ballots from notShuffleBallots, including an index of less than zero and more than the number of ballots.	notShuffleBallots	Throw an out-of-range error.	The function throws an out-of-range error as expected on all cases.	None

Post condition(s) for Test:

Each ballot retrieved by index in notShuffleBallots matches the expected one from notShuffleBallotsVector.
System state remains unchanged.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/10**Test Case ID#:** 4**Name(s) of Testers:** Mark Tsai**Test Description:**

Ballots class getCandidates function test.

The test is in testBallots.cpp with the test name

GetCandidateTest. The Ballots getCandidates function is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.**Automated:** yes**Results:** Pass**Preconditions for Test:**

Assume the ballot files do not have data errors based on the project description, no exception handling for the ballots needed.

A candidate vector containing the candidate names, notShuffleCandidatesVector, is created, and a ballot vector that contains the corresponding ballot information, notShuffleBallotsVector, is created. These two vectors are used to create a Ballots class object, notShuffleBallots, with the Ballots class constructor and shuffle option "false".

A candidate vector containing the candidate names, shuffleCandidatesVector, is created, and a ballot vector that contains the corresponding ballot information, shuffleBallotsVector, is created. These two vectors are used to create a Ballots class object, shuffleBallots, with the Ballots class constructor and shuffle option "true".

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Retrieve candidate name vector from notShuffleBallots and compared it to notShuffleCandidatesVector	notShuffleBallots notShuffleCandidatesVector	Two vectors should be the same.	Two vectors are the same.	None
2	Retrieve candidate name vector from shuffleBallots and compared it to shuffleCandidatesVector	shuffleBallots shuffleCandidatesVector	Two vectors should be the same.	Two vectors are the same.	None

Post condition(s) for Test:

The candidate vector returned by getCandidates, regardless of whether the ballots are shuffled or not, matches the input candidate name vector.

System state remains unchanged.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/10**Test Case ID#:** 5**Name(s) of Testers:** Mark Tsai**Test Description:**

Ballots class getBallotCount function test.

The test is in testBallots.cpp with the test name

GetBallotCountTest. The Ballots getBallotCount function is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.**Automated:** yes**Results:** Pass**Preconditions for Test:**

Assume the ballot files do not have data errors based on the project description, no exception handling for the ballots needed.

A candidate vector contains the candidate names, notShuffleCandidatesVector, is created, and a ballot vector that contains the corresponding ballot information, notShuffleBallotsVector, is created. These two vectors are used to create a Ballots class object, notShuffleBallots, with the Ballots class constructor and shuffle option “false”.

A candidate vector contains the candidate names, shuffleCandidatesVector, is created, and a ballot vector that contains the corresponding ballot information, shuffleBallotsVector, is created. These two vectors are used to create a Ballots class object, shuffleBallots, with the Ballots class constructor and shuffle option “true”.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Retrieve ballot count from notShuffleBallots and compared to size of notShuffleBallotsVector	notShuffleBallots notShuffleBallotsVector	The number should be the same as the size of the vector.	The two numbers are the same.	None
2	Retrieve ballot count from shuffleBallots and compared to size of shuffleBallotsVector	shuffleBallots shuffleBallotsVector	The number should be the same as the size of the vector.	The two numbers are the same.	None

Post condition(s) for Test:

The ballot count returned by getBallotCount, regardless of whether the ballots are shuffled or not, matches the input ballot vector size. System state remains unchanged.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/10**Test Case ID#:** 6**Name(s) of Testers:** Mark Tsai**Test Description:**

Ballots class getCandidateCount function test.

The test is in testBallots.cpp with the test name

GetCandidateCountTest. The Ballots getCandidateCount function is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.**Automated:** yes**Results:** Pass**Preconditions for Test:**

Assume the ballot files do not have data errors based on the project description, no exception handling for the ballots needed.

A candidate vector contains the candidate names, notShuffleCandidatesVector, is created, and a ballot vector that contains the corresponding ballot information, notShuffleBallotsVector, is created. These two vectors are used to create a Ballots class object, notShuffleBallots, with the Ballots class constructor and shuffle option "false".

A candidate vector contains the candidate names, shuffleCandidatesVector, is created, and a ballot vector that contains the corresponding ballot information, shuffleBallotsVector, is created. These two vectors are used to create a Ballots class object, shuffleBallots, with the Ballots class constructor and shuffle option "true".

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Retrieve candidate count from notShuffleBallots and compared to size of notShuffleCandidatesVector	notShuffleBallots notShuffleCandidatesVec tor	The number should be the same as the size of the vector.	The two numbers are the same.	None
2	Retrieve candidate count from shuffleBallots and compared to size of shuffleCandidatesVector	shuffleBallots shuffleCandidatesVector	The number should be the same as the size of the vector.	The two numbers are the same.	None

Post condition(s) for Test:

The candidate count returned by getCandidateCount, regardless of whether the ballots are shuffled or not, matches the input candidate vector size.

System state remains unchanged.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/10**Test Case ID#:** 7**Name(s) of Testers:** Mark Tsai**Test Description:**

Ballots class getShuffle function test.

The test is in testBallots.cpp with the test name GetShuffleTest.

The Ballots getShuffle function is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.**Automated:** yes**Results:** Pass**Preconditions for Test:**

Assume the ballot files do not have data errors based on the project description, no exception handling for the ballots needed.

A candidate vector contains the candidate names, notShuffleCandidatesVector, is created, and a ballot vector that contains the corresponding ballot information, notShuffleBallotsVector, is created. These two vectors are used to create a Ballots class object, notShuffleBallots, with the Ballots class constructor and shuffle option "false".

A candidate vector contains the candidate names, shuffleCandidatesVector, is created, and a ballot vector that contains the corresponding ballot information, shuffleBallotsVector, is created. These two vectors are used to create a Ballots class object, shuffleBallots, with the Ballots class constructor and shuffle option "true".

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Retrieve shuffle option from notShuffleBallots and compared to expected value "false".	notShuffleBallots	The shuffle option value should be "false".	The shuffle option value is "false".	None
2	Retrieve shuffle option from shuffleBallots and compared to expected value "true".	shuffleBallots	The shuffle option value should be "true".	The shuffle option value is "true".	None

Post condition(s) for Test:

The shuffle option value is the same as indicated when the Ballots object is created.

System state remains unchanged.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/10**Test Case ID#:** 8**Name(s) of Testers:** Mark Tsai**Test Description:**

Candidate class assignBallot function test.

The test is in testCandidate.cpp with the test name

AssignBallotTest. The Candidate assignBallot function is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.**Automated:** yes**Results:** Pass**Preconditions for Test:**

Assume the ballot files do not have data errors based on the project description, no exception handling for the ballots needed.

A vector containing ballot IDs, idVector, is created. A string contains "Manan", name, is created. And a Candidate object, oneCandidate, is created with the name.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Assign each ballot ID in idVector to oneCandidate and push back to a temp vector to check whether the assignedBallots attribute contains those assigned IDs in the temp vector.	oneCandidate idVector	The current assigned IDs to oneCandidate should be the same as the ones in the temp vector.	The current assigned IDs to oneCandidate are the same as the ones in the temp vector.	None

Post condition(s) for Test:

The oneCandidate has the same assigned IDs from idVector in its assignedBallots attribute as the temp vector that stores the same current IDs.

System state remains unchanged.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/10**Test Case ID#:** 9**Name(s) of Testers:** Mark Tsai**Test Description:**

Candidate class getAssignedBallots function test.
The test is in testCandidate.cpp with the test name
GetAssignBallotTest. The Candidate getAssignedBallots function
is being used.

**Indicate where are you storing the tests (what file) and the
name of the method/functions being used.**

Automated: yes**Results:** Pass**Preconditions for Test:**

Assume the ballot files do not have data errors based on the project description, no exception handling for the ballots needed.
A vector containing ballot IDs, idVector, is created. A string contains "Manan", name, is created. And a Candidate object,
oneCandidate, is created with the name.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Assign each ballot ID in idVector to oneCandidate and retrieve the assignedBallots attribute with getAssignedBallots to check whether it is the same as idVector.	oneCandidate idVector	The getAssignedBallots should return the same vector as idVector.	The getAssignedBallots returns the same vector as idVector.	None

Post condition(s) for Test:

The getAssignedBallots returns the assignedBallots attribute, which is the same vector as idVector.
System state remains unchanged.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/10**Test Case ID#:** 10**Name(s) of Testers:** Mark Tsai**Test Description:**

Candidate class getBallotNum function test.

The test is in testCandidate.cpp with the test name

GetBallotNumTest. The Candidate getBallotNum function is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.**Automated:** yes**Results:** Pass**Preconditions for Test:**

Assume the ballot files do not have data errors based on the project description, no exception handling for the ballots needed.

A vector containing ballot IDs, idVector, is created. A string contains "Manan", name, is created. And a Candidate object, oneCandidate, is created with the name.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Assign each ballot ID in idVector to oneCandidate and retrieve number of assigned ballots with getBallotNum function.	oneCandidate idVector	The number that getBallotNum returns should be the same as the size of idVector.	Two numbers are the same.	None

Post condition(s) for Test:

The getBallotNum returns the number of ballots assigned to oneCandidate, which is the same as the size of idVector.

System state remains unchanged.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/10**Test Case ID#:** 11**Name(s) of Testers:** Mark Tsai**Test Description:**

Candidate class getName function test.

The test is in testCandidate.cpp with the test name GetNameTest.

The Candidate getName function is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.**Automated:** yes**Results:** Pass**Preconditions for Test:**

Assume the ballot files do not have data errors based on the project description, no exception handling for the ballots needed.

A vector containing ballot IDs, idVector, is created. A string contains "Manan", name, is created. And a Candidate object, oneCandidate, is created with the name.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Retrieve the name of oneCandidate with the function getName and check whether it is the same as the name variable or not.	oneCandidate name (the string variable contains "Manan")	The retrieved string should be the same as the name variable.	Two strings are the same.	None

Post condition(s) for Test:The getName returns the name stored in oneCandidate, which is the same as the name variable that contains "Manan".
System state remains unchanged.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/04/16**Test Case ID#:** 12 - refactor main**Name(s) of Testers:** Mark Tsai**Test Description:**

Driver class shuffleOffFlag function test.

The test is in testDriver.cpp with the test name ShuffleFlagTest.

The function in Driver.cpp, shuffleOffFlag, is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.**Automated:** yes**Results:** Pass**Preconditions for Test:**

Sets of argc and argv used in the test are created. (good_argc, good_argv) contains just the program name. (good_shuffle_argc, good_shuffle_argv) contains the program name and the correct shuffle-off flag. (bad_argc, bad_argv) contains the program name and the wrong shuffle-off flag. (many_argc, many_argv) contains the program name, the correct shuffle-off flag, and one extra argument.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Test the function with (good_argc, good_argv) and the result shuffle variable	good_argc good_argv	The result shuffle variable should be set to true, and the function returns 0.	The result shuffle variable is set to true, and the function returns 0.	None
2	Test the function with (good_shuffle_argc, good_shuffle_argv) and the result shuffle variable	good_shuffle_argc good_shuffle_argv	The result shuffle variable should be set to false, and the function returns 0.	The result shuffle variable is set to false, and the function returns 0.	None
3	Test the function with (bad_argc, bad_argv) and the result shuffle variable	bad_argc bad_argv	The result shuffle variable should be set to true, and the function returns 1.	The result shuffle variable is set to true, and the function returns 1.	None

4	Test the function with (many_argc, many_argv) and the result shuffle variable	many_argc many_argv	The result shuffle variable should be set to true, and the function returns 1.	The result shuffle variable is set to true, and the function returns 1.	None
---	---	------------------------	--	---	------

Post condition(s) for Test:

The shuffleOffFlag flag function, by default, sets the shuffle variable to true if no shuffle-off flag is taken, and sets it to false if a correct shuffle-off flag is provided. If too many arguments or the wrong flag is provided, the function returns 1 to indicate an error. Otherwise, it returns 0 to indicate a successful run.

System state remains unchanged.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/04/16**Test Case ID#:** 13 - refactor main**Name(s) of Testers:** Mark Tsai**Test Description:**

FileHandler class open_file function test.

The test is in testFileHandler.cpp with the test name

OpenFileTest. The function in FileHandler.cpp, open_file, is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.**Automated:** yes**Results:** Pass**Preconditions for Test:**

Assume the ballot files do not have data errors based on the project description, no exception handling for the ballots needed.

A vector of inputs for bad file names, bad_file_name, is created. The bad inputs include wrong file name, no csv extension, and wrong directory. It contains mainly the wrong file name, but the last one is the correct one, because then the function won't be stuck in an infinite loop.

Vectors of input for good file name for plurality, plurality_file_name, and for STV, stv_file_name, are created.

A function userInput takes these vectors and simulates user input to the function by redirecting STDIN_FILENO and a function, restore_stdin_fd, to restore the STDIN_FILENO.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Redirect and simulate the user input with bad_file_name. Call open_file function. Check whether the file is opened, close the file, and restore the input redirection.	bad_file_name ../testing/stvTestFileHandler.csv	The file is opened.	The file is opened.	The last input in bad_file_name is the correct one, so that the function won't be stuck in an infinite loop.
2	Redirect and simulate the user input with plurality_file_name. Call open_file function. Check whether the file is	plurality_file_name ../testing/pluralityTestFileHandler.csv	The file is opened.	The file is opened.	None

	opened, close the file, and restore the input redirection.				
3	Redirect and simulate the user input with stv_file_name. Call open_file function. Check whether the file is opened, close the file, and restore the input redirection.	stv_file_name ../testing/stvTestFileHan dler.csv	The file is opened.	The file is opened.	None

Post condition(s) for Test:

The function will keep prompting the user to enter the correct file name if the wrong file name is provided. And if a correct file name is provided, the file would be opened.

System state remains unchanged.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/04/16**Test Case ID#:** 14 - refactor main**Name(s) of Testers:** Mark Tsai**Test Description:**

FileHandler class read_file function test.

The test is in testFileHandler.cpp with the test name

ReadFileTest. The function in FileHandler.cpp, read_file, is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.**Automated:** yes**Results:** Pass**Preconditions for Test:**

Assume the ballot files do not have data errors based on the project description, no exception handling for the ballots needed.

Vectors of input for good file name for plurality, plurality_file_name, and for STV, stv_file_name, are created.

A vector, ballots_stv, containing the same ballot information as ../testing/pluralityTestFileHandler.csv, ballots_plurality, and a vector containing the same ballot information as ../testing/stvTestFileHandler.csv.

A function userInput takes these vectors and simulates user input to the function by redirecting STDIN_FILENO and a function, restore_stdin_fd, to restore the STDIN_FILENO.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Redirect and simulate the user input with plurality_file_name. Call open_file function. Call read_file function and store the result Ballots object. Close the file, and restore the input redirection.	plurality_file_name ballots_plurality candidates ../testing/pluralityTestFileHandler.csv	The ballot vector filled in by the function should be the same as the ballots_plurality, and the candidate vector should be the same as the candidates	The corresponding vector filled in by the function has the same value as the expected vector.	None
2	Redirect and simulate the user input with stv_file_name. Call open_file function. Call read_file function and	stv_file_name ballots_stv candidates ../testing/stvTestFileHandler.csv	The ballot vector filled in by the function should be the same as the ballots_plurality, and the candidate vector should	The corresponding vector filled in by the function has the same value as the expected vector.	None

	store the result Ballots object. Close the file, and restore the input redirection.		be the same as the candidates		
--	---	--	-------------------------------	--	--

Post condition(s) for Test:

The function returns the Ballots object with the same ballot information as stored in the opened file, regardless of whether it is for plurality or STV.

System state remains unchanged.

Project Name: Project 2: Voting System

Team# 11

Test Stage: Unit

Test Date: 2025/03/27

Test Case ID#: 15

Name(s) of Testers: Andy Dang

Test Description:

Plurality constructor test. Located in testPlurality.cpp.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes

Results: Pass

Preconditions for Test:

The seats and ballots arguments exist and are valid.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Create an instance of the Plurality class	ballots_plurality, seats = 2	The member attributes should be initialized correctly.	The member attributes are initialized correctly.	None

Post condition(s) for Test:

An instance of the Plurality class has been created. The state of the system has not been changed.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/27**Test Case ID#:** 16**Name(s) of Testers:** Andy Dang**Test Description:**

Runs a simple plurality election. Located in testPlurality.cpp.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.**Automated:** yes**Results:** Pass**Preconditions for Test:**

The Plurality constructor works properly, and the member attributes are initialized accordingly

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Runs a plurality election	ballots_plurality, seats = 2	Bill and Alice should win the election	Bill and Alice won the election	None
2	Runs a plurality election with one candidate	ballots_plurality_four, seats = 3	Andy should win the election	Andy won the election	None
3	Runs a plurality election with one seat	ballots_plurality, seats = 1	Bill should win the election	Bill won the election	None
4	Runs a plurality election with one candidate and one seat	ballots_plurality_four, seats = 1	Andy should win the election	Andy won the election	None

Post condition(s) for Test:

A plurality election has been processed, with Bill and Alice being winners. The system is now in a state where an election has been processed.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/27**Test Case ID#:** 17**Name(s) of Testers:** Andy Dang**Test Description:**

Test for the output of a plurality election. Located in testPlurality.cpp.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes**Results:** Pass**Preconditions for Test:**

The Plurality constructor works properly, and the member attributes are initialized accordingly

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Create an instance of the Plurality class	ballots_plurality, seats = 2	Election type, # of seats, # of ballots, # of candidates, winners, and losers should be displayed, with each candidate having the correct percentage of ballots.	vElection type, # of seats, # of ballots, # of candidates, winners, and losers are displayed, with each candidate having the correct percentage of ballots.	The function that is tested, displayElectionDetails(), is called inside runElection()

Post condition(s) for Test:

A plurality election has been processed, and the results are displayed on the terminal. The system is now in a state where an election has been processed.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/27**Test Case ID#:** 18**Name(s) of Testers:** Andy Dang**Test Description:**

Time the plurality election with 100,000 ballots. Located in testPlurality.cpp.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes**Results:** Pass**Preconditions for Test:**

None

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Time the process of creating ballots, creating a Plurality instance, and running an election	ballots_plurality_two, candidates_two seats = 5	The time it takes to process the election should be under 5 minutes.	The time it takes to process the election is under 5 minutes.	None

Post condition(s) for Test:

A plurality election has been processed in under 5 minutes. The state of the system has not been changed.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/27**Test Case ID#:** 19**Name(s) of Testers:** Andy Dang**Test Description:**

Tests the randomizer if there are multiple winning candidates for one seat. Located in testPlurality.cpp.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes / no**Results:** Pass**Preconditions for Test:**

The seats and ballots arguments exist and are valid.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Run a plurality election with a ballot that has 2 candidates tying for the final seat	ballots_plurality_three, seats = 3	Bill and Alice should be winners, and Siyang and Preeti should be losers	Bill and Alice are winners, and Siyang and Preeti are losers	None
2	Repeat Step 1 multiple times (at least 10)	ballots_plurality_three, seats = 3	Ahmed should win around half the time, and Sally should win around half the time	Ahmed wins around half the time, and Sally wins around half the time	Run ./testPlurality multiple times to test the randomizer (tiebreaker)
3	Run the same plurality election multiple times (at least 10) that has more than 2 candidates tying for the final seat	ballots_plurality_seven, seats = 1	Manan should win $\frac{1}{3}$ of the time, Mark should win $\frac{1}{3}$ of the time, Michael should win $\frac{1}{3}$ of the time	Manan won 5 times, Mark won 4 times, Michael won once	This was done in the same way as Step 2
4	Run the same plurality election multiple times (at least 10) that has everyone tying for the final seat	ballots_plurality_eight, seats = 1	Each of the candidates is expected to win once	4 candidates won once, 2 won twice, and 2 never won	This was done in the same way as Step 2

Post condition(s) for Test:

Multiple plurality elections have been processed, with Bill and Alice being winners, and Siyang and Preeti losers. Ahmed is expected to win half the time, and Sally is expected to win half the time. The system is now in a state where an election has been processed.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/28**Test Case ID#:** 20**Name(s) of Testers:** Michael Dunn**Test Description:**

Multiple candidates are directly elected by the STV election without needing winner or loser vote redistribution.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

The seats, ballots, and audit filename arguments exist and are valid.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Run an STV election with a ballot that has 2 candidates meeting the Droop quota and being selected as winners.	basic_win, seats = 2.	Bill Jones and Sally Ride should be winners, and Alice Mix, Ahmed Mohamed, Siyang Xiong, and Preeti Banerjee should be losers. Both lists should be returned in that order.	Bill Jones and Sally Ride were selected as winners, and Alice Mix, Ahmed Mohamed, Siyang Xiong, and Preeti Banerjee were selected as losers.	

Postcondition(s) for Test:

Election details and audit files match the expected election outcome.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/28**Test Case ID#:** 21**Name(s) of Testers:** Michael Dunn**Test Description:**

Testing that when a winner hits the Droop quota and is moved to the winners' list, any additional ballots they receive must immediately be redistributed to the next available option.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

The seats, ballots, and audit filename arguments exist and are valid.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Run an STV election, and when distributing a ballot that would've gone to a candidate already in the winners' list, assign it instead the ballot's next option, causing another candidate to reach the Droop quota.	winner_reassign_win, seats = 2.	Bill Jones and Sally Ride should be winners, and Alice Mix, Ahmed Mohamed, Siyang Xiong, and Preeti Banerjee should be losers. Both lists should be returned in that order.	Bill Jones and Sally Ride were selected as winners, and Alice Mix, Ahmed Mohamed, Siyang Xiong, and Preeti Banerjee were selected as losers.	

Postcondition(s) for Test:

Election details and audit files match the expected election outcome. The ballot that was reassigned is assigned to one candidate and therefore only shows up once.

Election details and audit files match the expected election outcome.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/28**Test Case ID#:** 22**Name(s) of Testers:** Michael Dunn**Test Description:**

Testing the case that all ballots are distributed and there are still seats that need to be filled. The candidate with the fewest (non-zero) votes has their votes redistributed, which causes a candidate that hasn't 'lost' yet to hit the Droop quota, and the remaining seat being filled.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

The seats, ballots, and audit filename arguments exist and are valid.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Run an STV election and distribute all votes to their respective candidates. After votes are distributed, find the candidate with the lowest votes and redistribute them, which elects the final candidate.	loser_reassign_win, seats = 2.	Bill Jones will hit the Droop quota and be selected as a winner, and Sally Ride will receive a redistributed ballot from Siyang Xiong, allowing them to hit the Droop quota.	Bill Jones and Sally Ride were selected as winners, and Alice Mix, Ahmed Mohamed, Preeti Banerjee, and Siyang Xiong, were selected as losers. Siyang Xiong's redistributed ballot was successfully removed from their assigned ballots list and reassigned to Sally Ride.	

Postcondition(s) for Test:

Election details and audit files match the expected election outcome. The ballot that was reassigned is assigned to one candidate and therefore only shows up once.

Election details and audit files match the expected election outcome.

Project Name: Project 2: Voting System

Team# 11

Test Stage: Unit

Test Date: 2025/03/28

Test Case ID#: 23

Name(s) of Testers: Michael Dunn

Test Description:

Testing the display of election details after an election is run.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes

Results: Pass

Preconditions for Test:

The seats, ballots, and audit filename arguments exist and are valid.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Finish an STV election and ensure that the correct details are output to the screen.	basic_win, seats = 2.	Election type: STV Number of seats: 2 Number of ballots: 6 Number of candidates: 6 Winners: Bill Jones Sally Ride Losers: Alice Mix Ahmed Mohamed Siyang Xiong Preeti Banerjee	Election type: STV Number of seats: 2 Number of ballots: 6 Number of candidates: 6 Winners: Bill Jones Sally Ride Losers: Alice Mix Ahmed Mohamed Siyang Xiong Preeti Banerjee	

Postcondition(s) for Test:

The correct election details are printed on the screen.

Election details and audit files match the expected election outcome.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/28**Test Case ID#:** 24**Name(s) of Testers:** Michael Dunn

Test Description: In the case that two candidates receive the same number of votes, and there is only one seat to be elected, the STV election will need to choose between the two candidates randomly. This test ensures that only the two candidates with the same number of votes are considered for the seat.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

The seats, ballots, and audit filename arguments exist and are valid.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	After the ballots are distributed to their first choice candidate in an STV election, there is a tie between two candidates. One of the tied candidates and their votes will be redistributed, leading to the other candidate to win.	basic_win, seats = 1.	Sally Ride or Bill Jones hit the Droop quota and wins the election.	The losers are Alice Mix, Ahmed Mohamed, Siyang Xiong, Preeti Banerjee, and either Bill Jones or Sally Ride are selected as losers in that order.	

Postcondition(s) for Test:

The correct election details are printed on the screen.

Election details and audit files match the expected election outcome.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** System**Test Date:** 2025/04/21**Test Case ID#:** 25**Name(s) of Testers:** Manan Chaturvedi

Test Description: Testing that the STV algorithm runs properly under the event that there are a surplus of candidates and seats, but a small amount of ballots. Tests the Driver, FileHandler, and STV classes.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

A valid csv file with a header of the algorithm type (in this case STV), with the number of seats, candidates, and ballots.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	The csv file and audits are inputted by the user, then the election is run.	CSV File and Audit Text file are inputted (audit file name after election.)	A valid election class along with the audit file.	An election class along with with an audit file named "lowVoteValid.txt"	
2	The results from the election that are printed onto the screen are captured by a string and compared to the exact answer that should come out.	The string taken from the election is compared to the string assumed by the results of the election run properly such that there is only one winner and 19 losers.	The results is that both the election and expected strings are the same.	Such is proven.	

Postcondition(s) for Test:

The election is ran correctly.

Audit file showcases the correct election process and results.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** System**Test Date:** 2025/03/29**Test Case ID#:** 26**Name(s) of Testers:** Manan Chaturvedi, Andy Dang

Test Description: Testing that the PV algorithm runs properly under the event that there are a surplus of candidates and seats, but a small amount of ballots. Tests the Driver, FileHandler, and Plurality classes.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

A valid csv file with a header of the algorithm type (in this case Plurality), with the number of seats, candidates, and ballots.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	The csv file and audits are inputted by the user, then the election is run.	CSV File is inputted	A valid election clas.	An election class is returned.	
2	The results from the election that are printed onto the screen are captured by a string and compared to the exact answer that should come out.	The string taken from the election is compared to the string assumed by the results of the election run properly such that every candidate wins, along with the proper percentages for each candidate.	The results is that both the election and expected strings are the same.	Such is proven.	

Postcondition(s) for Test:

Election details match the expected election outcome.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** System**Test Date:** 2025/03/29**Test Case ID#:** 27**Name(s) of Testers:** Manan Chaturvedi, Andy Dang

Test Description: Testing that the PV algorithm runs properly under the event that there are a surplus of candidates and seats, but a large amount of ballots. Tests the Driver, FileHandler, and Plurality classes.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

A valid csv file with a header of the algorithm type (in this case Plurality), with the number of seats, candidates, and ballots.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	The csv file and audits are inputted by the user, then the election is run.	CSV File is inputted	A valid election clas.	An election class is returned.	
2	The results from the election that are printed onto the screen are captured by a string and compared to the exact answer that should come out.	The string taken from the election is compared to the string assumed by the results of the election run properly such that every candidate wins, along with the proper percentages for each candidate.	The results is that both the election and expected strings are the same.	Such is proven.	

Postcondition(s) for Test:

Election details match the expected election outcome.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** System**Test Date:** 2025/03/29**Test Case ID#:** 28**Name(s) of Testers:** Manan Chaturvedi

Test Description: Testing that the STV algorithm runs properly under the event that there are a surplus of candidates and seats, but a large amount of ballots. Tests the Driver, FileHandler, and STV classes.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

A valid csv file with a header of the algorithm type (in this case STV), with the number of seats, candidates, and ballots.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	The csv file and audits are inputted by the user, then the election is run.	CSV File and Audit Text file are inputted (audit file name after election.)	A valid election class along with the audit file.	An election class along with with an audit file named "seatValid.txt"	
2	The results from the election that are printed onto the screen are captured by a string and compared to the exact answer that should come out.	The string taken from the election is compared to the string assumed by the results of the election run properly such that there is only one winner and 19 losers.	The results is that both the election and expected strings are the same.	Such is proven.	

Postcondition(s) for Test:

Election details and audit files match the expected election outcome.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** System**Test Date:** 2025/03/29**Test Case ID#:** 29**Name(s) of Testers:** Manan Chaturvedi

Test Description: Tests that the shuffling of the ballots is performed fairly and properly such that no ballots, when shuffled, are unique to the original ballot. This test uses the Driver, FileHanlder, Plurality, and Ballots class.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

A valid csv file with a header of the algorithm type (in this case Plurality), with the number of seats, candidates, and ballots.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	A Plurality election is ran with the shuffle flag truend off initially to have a control group.	An election class is returned to use as reference, along with the total ballot count.	No direct results other than election and ballot count.	No direct results other than election and ballot count.	
2	Another plurality election is ran - all ballots from both elections are compared and stored in an integer - this is ran and added 1000 times.	A new election class with shuffled ballots, along with an integer containing all of the ballots not equal to one another.	No direct results other than election, along with total ballots that are shuffled.	No direct results other than election, along with total ballots that are shuffled.	
3	The total number of shuffled ballots is divided by 1000 to get the total number of shuffled ballots.	There exists an integer that contains the total ballots that were shuffled from all ballot comparisons.	The total ballot shuffle percentage is above 75%.	The actual value ranges from 78 to 80% on average.	This is ran on 100 ballot elections, so the value is divided cleanly by 1000 based on the number of tests we run.

Postcondition(s) for Test:

Elections are valid.

Ballots are shuffled to a degree above 75% - shuffled ballots in elections are shuffled properly.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** System**Test Date:** 2025/03/29**Test Case ID#:** 30**Name(s) of Testers:** Manan Chaturvedi

Test Description: Tests that the STV election runs fairly based on the type of ballot given - results are the same if the ballot is shuffled or not. Uses the Driver, FileHandler, and STV class.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

A valid csv file with a header of the algorithm type (in this case STV), with the number of seats, candidates, and ballots.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	A STV election is ran with an audit file created to store the relevant information with.	Creates an election class from a csv with the audit file named "noShuffle.txt".	No expected results.	No expected results.	
2	Another STV election is ran with the shuffle mode turned on with another audit text file.	Creates an election class from the same csv file with the audit file named "withShuffle.txt".	No expected results.	No expected results.	
3	We take the winners from each ballot and get each of the assigned ballots from each of them and their size. Then we determine that the winners from both sides are the same.	Creates winnerCandidate vectors with shuffled and non shuffled values, along with assigned ballot sizes for each associated vector.	Each vector associated with winners should be the same with the assigned ballots.	Such is the case that the winner sizes are the same.	Names are unreliable due to the random choice from a tie, so using the ballots is a better way to determine validity.

Postcondition(s) for Test:

Election details and audit files match the expected election outcome.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** System**Test Date:** 2025/03/29**Test Case ID#:** 31**Name(s) of Testers:** Manan Chaturvedi

Test Description: Tests that the Plurality election runs fairly based on the type of ballot given - results are the same if the ballot is shuffled or not. Uses the Driver, FileHandler, and Plurality class.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

A valid csv file with a header of the algorithm type (in this case Plurality), with the number of seats, candidates, and ballots.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	A Plurality election is ran.	Creates an election class from a csv.	No expected results.	No expected results.	
2	Another Plurality election is ran with the shuffle mode turned on.	Creates an election class from the same csv file.	No expected results.	No expected results.	
3	We take the winners and losers from each ballot and get each of the assigned ballots from each of them and their size. Then we determine that the winners and losers from both sides are the same along with the percentages.	Creates winner and loser Candidate vectors with shuffled and non shuffled values, along with the proper placement and percentages made for each candidate.	Each vector associated with winners and losers should be the same with the assigned percentages.	Such is the case that the winners and losers sizes are the same with the right percentages.	

Postcondition(s) for Test:

Election details and audit files match the expected election outcome.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** System**Test Date:** 2025/03/29**Test Case ID#:** 32**Name(s) of Testers:** Manan Chaturvedi

Test Description: Tests that the STV election runs accurately based on the smallest possible inputs. - results are the same if the ballot is shuffled or not. Uses the Driver, FileHandler, and STV class.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

A valid csv file with a header of the algorithm type (in this case STV), with the number of seats, candidates, and ballots set to 1.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	A STV election is ran with an audit file created to store the relevant information with.	Creates an election class from a csv with the audit file named "minSTV.txt".	No expected results.	No expected results.	
2	The total number of winners, losers, and candidates are tested along with the valid audit file.	Creates integers related to the sizes of the winners, losers, and candidates.	The winners should be equal the the number of candidates (1) and the losers should be set to 0.	Winners are set to 1 and the losers are set to 0.	

Postcondition(s) for Test:

Election details and audit files match the expected election outcome.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** System**Test Date:** 2025/03/29**Test Case ID#:** 33**Name(s) of Testers:** Manan Chaturvedi

Test Description: Tests that the Plurality election runs accurately based on the smallest possible inputs. - results are the same if the ballot is shuffled or not. Uses the Driver, FileHandler, and Pluralityclass.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

A valid csv file with a header of the algorithm type (in this case Plurality), with the number of seats, candidates, and ballots set to 1.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	A Plurality election is ran.	Creates an election class from a csv file.	No expected results.	No expected results.	
2	The total number of winners, losers, and candidates are tested.	Creates integers related to the sizes of the winners, losers, and candidates.	The winners should be equal the number of candidates (1) and the losers should be set to 0.	Winners are set to 1 and the losers are set to 0.	

Postcondition(s) for Test:

Election details match the expected election outcome.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** System**Test Date:** 2025/03/29**Test Case ID#:** 34**Name(s) of Testers:** Manan Chaturvedi, Andy Dang

Test Description: This test just runs a basic Plurality election, but confirms that the winners and losers are accurate to the number of seats and total seats along with each winner having more votes than all of the losers. Tests the Driver, FileHandler, and Plurality classes.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

A valid csv file with a header of the algorithm type (in this case Plurality), with the number of seats, candidates, and ballots.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	A Plurality election is ran.	Creates an election class from a csv file.	No expected results.	No expected results.	
2	The total number of winners, losers, and candidates are tested.	Creates integers related to the sizes of the winners, losers, and candidates.	The winners should be equal the the number of seats and the losers should be set to the number of candidates minus seats.	Winners, losers, candidates, and seats are set to the right values.	
3	The winners and losers are all compared such that the winners ballot numbers are greater than the losers.	No test data is created, but we get the ballot numbers from each winner and loser.	All winners should have their ballot numbers be greater than all of the losers.	All winners have ballot numbers greater than all losers.	

Postcondition(s) for Test:

Election details match the expected election outcome.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** System**Test Date:** 2025/03/28**Test Case ID#:** 35**Name(s) of Testers:** Manan Chaturvedi, Andy Dang

Test Description: This test just runs a basic Plurality election, however there is a tie. This confirms that the winners and losers are accurate to the number of seats and total seats along with each winner having more votes than all of the losers. Tests the Driver, FileHandler, and Plurality classes.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

A valid csv file with a header of the algorithm type (in this case Plurality), with the number of seats, candidates, and ballots where a tie occurs within the election.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	A Plurality election is ran.	Creates an election class from a csv file.	No expected results.	No expected results.	Ties in plurality elections add a factor of inconsistency on hard-coded test results.
2	The values printed to the screen are all tested.	Tests the string printed to the string along with the values of the winners and losers.	The winners should be equal the the number of seats and the losers should be set to the number of candidates minus seats.	Winners, losers, candidates, and seats are set to the right values.	The string itself is tested to determine the seats, ballots, and candidates.
3	The winners and losers are all compared such that the winners ballot numbers are the same as the losers.	No test data is created, but we get the ballot numbers from each winner and loser.	All winners should have their ballot numbers be the same as all of the losers.	All winners have percentages that are the same as the losers.	

Postcondition(s) for Test:

The correct election details are printed on the screen.
Election details match the expected election outcome.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** System**Test Date:** 2025/03/29**Test Case ID#:** 36 - rework**Name(s) of Testers:** Manan Chaturvedi

Test Description: This test just runs a basic STV election, but confirms that the winners and losers are accurate to the number of seats and total seats along with each winner having more votes than all of the losers. Tests the Driver, FileHandler, and STV classes.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

A valid csv file with a header of the algorithm type (in this case STV), with the number of seats, candidates, and ballots.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	A STV election is ran with an audit file created to store the relevant information with.	Creates an election class from a csv with the audit file named "regSTV.txt".	No expected results.	No expected results.	
2	The total number of winners, losers, and candidates are tested along with the valid audit file.	Creates integers related to the sizes of the winners, losers, and candidates.	The winners should be equal the number of seats and the losers should be set to the number of candidates minus the seat numbers. The audit file should be accessible.	Winners are set to seat numbers and the losers are set to the candidates minus the seat numbers. The audit file is accessible.	
3	The winners and losers are all compared such that the winners ballot numbers are greater than the losers.	No test data is created, but we get the ballot numbers from each winner and loser.	All winners should have their ballot numbers be greater than all of the losers.	All winners have ballot numbers greater than all losers.	

Postcondition(s) for Test:

The correct election details are printed on the screen.

Election details and audit files match the expected election outcome.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** System**Test Date:** 2025/03/28**Test Case ID#:** 37**Name(s) of Testers:** Manan Chaturvedi

Test Description: This test just runs a basic STV election, however there is a tie. This confirms that the winners and losers are accurate to the number of seats and total seats along with each winner having more votes than all of the losers. Tests the Driver, FileHandler, and STV classes.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

A valid csv file with a header of the algorithm type (in this case STV), with the number of seats, candidates, and ballots where a tie occurs within the election.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	A STV election is ran with an audit file created to store the relevant information with.	Creates an election class from a csv with the audit file named "stvTie.txt".	No expected results.	No expected results.	
2	The total number of winners, losers, and candidates are tested along with the valid audit file.	Creates integers related to the sizes of the winners, losers, and candidates.	The winners should be equal the the number of seats and the losers should be set to the number of candidates minus the seat numbers. The audit file should be accessible.	Winners are set to seat numbers and the losers are set to the candidates minus the seat numbers. The audit file is accessible.	
3	The winners and losers are all compared such that the winners ballot numbers are greater or equal than the losers.	No test data is created, but we get the ballot numbers from each winner and loser.	All winners should have their ballot numbers be greater than all of the losers.	All winners have ballot numbers greater than all losers.	

Postcondition(s) for Test:

Election details and audit files match the expected election outcome.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** System**Test Date:** 2025/03/29**Test Case ID#:** 38**Name(s) of Testers:** Manan Chaturvedi

Test Description: This test determines the overall time that it takes for a STV election to perform a valid and verifiable election within a given amount of time. Tests the Driver, FileHandler, and STV classes.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

A valid csv file with a header of the algorithm type (in this case STV), with the number of seats, candidates, and ballots.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Starts a timer, but then performs a STV election as in previous tests - also checks validity using previous methods.	t0 = chrono::high_resolution::clock::now(); t1 = chrono::high_resolution::clock::now(); Using similar methods to run and determine the speed of the STV election given 100,000 ballots.	The time to perform a STV election is less than 300 seconds.	The time to perform a STV election is around 3 seconds.	100,000 ballots are chosen as the maximum value given the expectations of the program.

Postcondition(s) for Test:

Election details and audit files match the expected election outcome.

Project Name: Project 2: Voting System**Team# 11****Test Stage:** System**Test Date:** 2025/03/29**Test Case ID#:** 39**Name(s) of Testers:** Manan Chaturvedi, Andy Dang

Test Description: This test determines the overall time that it takes for a Plurality election to perform a valid and verifiable election within a given amount of time. Tests the Driver, FileHandler, and Plurality classes.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

A valid csv file with a header of the algorithm type (in this case Plurality), with the number of seats, candidates, and ballots.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Starts a timer, but then performs a Plurality election as in previous tests - also checks validity using previous methods.	t0 = chrono::high_resolution::clock::now(); t1 = chrono::high_resolution::clock::now(); string ret = testing::internal::GetCapturedStdout(); Corresponding vector<string> values (results, subWinners, subPercentages) to compare to.	The time to perform a Plurality election is less than 300 seconds.	The time to perform a Plurality election is less than 300 seconds.	

Postcondition(s) for Test:

Election details match the expected election outcome.

The PBI, the Task Description (from Sprint Log) with Unique Testing Number:

PBI: STV

Task Description: Refactor main unit test

Unique Testing Number: 40

Team Member(s) Responsible: Mark Tsai

Inputs:

plurality_file_name_header

ballots_plurality

../testing/pluralityWithHeaderTestFileHandler.csv

stv_file_name_header

ballots_stv

../testing/stvWithHeaderTestFileHandler.csv

candidates

Tests:

FileHandler class read_file overloaded version function test. The test is in testFileHandler.cpp with the test name ReadFileWithHeaderTest. The function in FileHandler.cpp, read_file, overloaded version with more arguments, is being used.

Outputs:

The ballot vector filled in by the function should contain the same ballots as expected corresponding vector.

The candidate vector filled in by the function should contain the same candidate names as candidates.

The algorithm type, seat number, candidate number, and ballot number, should be the same as the header in the file and corresponding to the information obtained from the file.

Passed or Failed: Passed

Date: 0417

The PBI, the Task Description (from Sprint Log) with Unique Testing Number:

PBI: STV

Task Description: Refactor main unit test

Unique Testing Number: 41

Team Member(s) Responsible: Mark Tsai

Inputs:

run_input

good_shuffle_argc

good_shuffle_argv

../testing/pluralityWithHeaderTestFileHandler.csv

Tests:

Driver class run function test. The test is in testDriver.cpp with the test name RunTest. The function in Driver.cpp, run, that in charge of running the election, is being used.

Outputs:

The output on screen should be as expected include the flow of user input and election result display for plurality.

Passed or Failed: Passed

Date: 0420

The PBI, the Task Description (from Sprint Log) with Unique Testing Number:

PBI: Input multiple files

Task Description: Unit testing

Unique Testing Number: 42

Team Member(s) Responsible: Mark Tsai

Inputs:

plurality_multiple_file_name

ballots_plurality_multiple

../testing/pluralityWithHeaderTestFileHandler.csv

../testing/pluralityWithHeaderTestFileHandler-2.csv

candidates

Tests:

FileHandler class multiple_files function test. The test is in testFileHandler.cpp with the test name MultipleFilesTest. The function in FileHandler.cpp, multiple_files, that can read multiple csv files, is being used.

Outputs:

The ballot vector filled in by the function should contain the same ballots as expected corresponding vector.

The candidate vector filled in by the function should contain the same candidate names as candidates.

The algorithm type, seat number, and candidate number, should be the same as the header in multiple files and corresponding to the information obtained from multiple files.

The ballot number should be the sum of all ballot number headers from multiple files and the same as the number of ballots in the ballot vector.

Passed or Failed: Passed

Date: 0422

The PBI, the Task Description (from Sprint Log) with Unique Testing Number:

PBI: Input multiple files

Task Description: Unit testing

Unique Testing Number: 43

Team Member(s) Responsible: Mark Tsai

Inputs:

run_multiple_file_input

good_shuffle_argc

good_shuffle_argv

../testing/pluralityWithHeaderTestFileHandler.csv

../testing/pluralityWithHeaderTestFileHandler-2.csv

Tests:

Driver class run function test. The test is in testDriver.cpp with the test name RunMultipleFileTest. The function in Driver.cpp, run, that in charge of running the election, is being used. This run test is testing the situation of multiple files input.

Outputs:

The output on screen should be as expected include the flow of user input and election result display for plurality.

Passed or Failed: Passed

Date: 0420

The PBI, the Task Description (from Sprint Log) with Unique Testing Number:

PBI: MV

Task Description: Unit testing (MV constructor test)

Unique Testing Number: 44

Team Member(s) Responsible: Andy Dang

Inputs:

municipal

ballots

seats = 3

candidates

ballots_municipal

Tests:

Create a Municipal object, and check that the member attributes are initialized properly

Outputs:

The ballots attribute of the Municipal object is the ballots input

The seats attribute is the same as the seats input

The winners and losers attributes are empty vectors

The candidates attribute is the candidates input, but with all elements initialized as Candidate objects

Passed or Failed: Passed

Date: 0423

The PBI, the Task Description (from Sprint Log) with Unique Testing Number:

PBI: MV

Task Description: Unit testing (run MV election test)

Unique Testing Number: 45

Team Member(s) Responsible: Andy Dang

Inputs:

municipal, ballots, seats = 3, candidates, ballots_municipal

municipal_two, ballots_two, seats = 1, candidates_two, ballots_municipal_two

municipal_four, ballots_four, seats = 1, candidates_four, ballots_municipal_four

municipal_five, ballots_five, seats = 21, candidates_five, ballots_municipal_five

municipal_six, ballots_six, seats = 2, candidates_six, ballots_municipal_six

Tests:

Run an MV election, check results for validity

Outputs:

Winners and losers vector have the correct size

Candidates have correct number of ballots assigned to them

Individual candidates not involved in ties are in correct vector (winners or losers)

Passed or Failed: Passed

Date: 0423

The PBI, the Task Description (from Sprint Log) with Unique Testing Number:

PBI: MV

Task Description: Unit testing (display MV details test)

Unique Testing Number: 46

Team Member(s) Responsible: Andy Dang

Inputs:

municipal

ballots

seats = 3

candidates

ballots_municipal

testing::internal::CaptureStdout()

Tests:

Run an MV election, then display the results on the screen

Outputs:

The first 89 characters of stdout are “Election type: Municipal\nNumber of seats: 3\nNumber of ballots: 9\nNumber of candidates: 6\n”

Passed or Failed: Passed

Date: 0423

The PBI, the Task Description (from Sprint Log) with Unique Testing Number:

PBI: MV

Task Description: Unit testing (MV algorithm execution time test)

Unique Testing Number: 47

Team Member(s) Responsible: Andy Dang

Inputs:

municipal_three

ballots_three

seats = 10

candidates_three

ballots_municipal_three

std::chrono::high_resolution_clock::now()

Tests:

Run an MV election, and time the execution time

Outputs:

The time it takes to run an MV election with 100000 ballots is less than 5 minutes

Passed or Failed: Passed

Date: 0423

The PBI, the Task Description (from Sprint Log) with Unique Testing Number:

PBI: MV

Task Description: System Testing

Unique Testing Number: 48

Team Member(s) Responsible: Manan Chaturvedi**Inputs:**

string file_name = "../testing/seatLowMV.csv"

Driver driver

Election* election

Ballots* ballots

Tests:

Test to run that MV works on an election with seats higher than the number of candidates but lower ballot count.

Compares output to expected results via string comparison.

Outputs:

string ret

vector<string> results

Passed or Failed: Passed**Date:** 0503

The PBI, the Task Description (from Sprint Log) with Unique Testing Number:

PBI: MV

Task Description: System Testing

Unique Testing Number: 49

Team Member(s) Responsible: Manan Chaturvedi**Inputs:**

string file_name = "../testing/MVSurplus.csv"

Driver driver

Election* election

Ballots* ballots

Tests:

Test to run that MV works on an election with seats higher than the number of candidates but higher ballot count.

Compares output to expected results via integer comparisons.

Outputs:

Municipal* MVelection

int winners_size

int losers_size

int candidates_size

Passed or Failed: Passed**Date:** 0503

The PBI, the Task Description (from Sprint Log) with Unique Testing Number:

PBI: MV

Task Description: System Testing

Unique Testing Number: 50

Team Member(s) Responsible: Manan Chaturvedi

Inputs:

string file_name = "../testing/MV4.csv"

Driver driver

Election* election

Ballots* ballots

Election* electionShuffle

Ballots* ballotsShuffle

Tests:

Test to run that MV runs fairly whether or not the election run is shuffled via the shuffle-off flag being toggled.

Compares output to expected results via string comparison.

Outputs:

string ret

vector<string> results

string retS

vector<string> resultsS

Passed or Failed: Passed

Date: 0503

The PBI, the Task Description (from Sprint Log) with Unique Testing Number:

PBI: MV

Task Description: System Testing

Unique Testing Number: 51

Team Member(s) Responsible: Manan Chaturvedi**Inputs:**

string file_name = "../testing/MV1.csv"

Driver driver

Election* election

Ballots* ballots

Tests:

Test to run that MV works with the smallest possible election size of 1 candidate, 1 seat, and 1 vote.

Compares output to expected results via integer comparisons.

Outputs:

Municipal* MVelection

int winners_size

int losers_size

int candidates_size

Passed or Failed: Passed**Date:** 0503

The PBI, the Task Description (from Sprint Log) with Unique Testing Number:

PBI: MV

Task Description: System Testing

Unique Testing Number: 52

Team Member(s) Responsible: Manan Chaturvedi**Inputs:**

string file_name = "../testing/MV3.csv"

Driver driver

Election* election

Ballots* ballots

Tests:

Test to run that MV works with a regular run with only one file.

Compares output to expected results via integer comparisons.

Outputs:

Municipal* MVelection

vector<Candidate> winners

vector<Candidate> losers

int seatNum

Passed or Failed: Passed**Date:** 0503

The PBI, the Task Description (from Sprint Log) with Unique Testing Number:

PBI: MV

Task Description: System Testing

Unique Testing Number: 53

Team Member(s) Responsible: Manan Chaturvedi**Inputs:**

string file_name = "../testing/MVTie.csv"

Driver driver

Election* election

Ballots* ballots

Tests:

Test to run that MV runs fairly in a scenario where the election results in a tie.

Compares output to expected results via string comparison

Outputs:

string ret

vector<string> results

Passed or Failed: Passed**Date:** 0503

The PBI, the Task Description (from Sprint Log) with Unique Testing Number:

PBI: MV

Task Description: System Testing

Unique Testing Number: 54

Team Member(s) Responsible: Manan Chaturvedi**Inputs:**

string file_name = "../testing/MV5.csv"

Driver driver

Election* election

Ballots* ballots

Tests:

Test to run that MV runs fairly in a scenario with the maximum number of randomly generated ballots.

Compares output to expected results via string comparison

Outputs:

string ret

vector<string> results

Passed or Failed: Passed**Date:** 0503

The PBI, the Task Description (from Sprint Log) with Unique Testing Number:

PBI: Input multiple files

Task Description: System Testing

Unique Testing Number: 55

Team Member(s) Responsible: Manan Chaturvedi

Inputs:

```
vector<string> file_names = {"../testing/Plurality6.csv", "../testing/Plurality7.csv",  
"../testing/Plurality8.csv", "../testing/Plurality9.csv", "../testing/Plurality10.csv"}
```

Driver driver

Election* election

Ballots* ballots

Tests:

Test to run that Plurality works with a regular run with multiple files inputed.

Compares output to expected results via integer comparisons.

Outputs:

Plurality* PVElection

vector<Candidate> winners

vector<Candidate> losers

int seatNum

Passed or Failed: Passed

Date: 0503

The PBI, the Task Description (from Sprint Log) with Unique Testing Number:

PBI: Input multiple files

Task Description: System Testing

Unique Testing Number: 56

Team Member(s) Responsible: Manan Chaturvedi

Inputs:

vector<string> file_names = {"../testing/STV6.csv", "../testing/STV7.csv",

"../testing/STV8.csv", "../testing/STV9.csv", "../testing/STV10.csv"}
string auditFile = "../testing/audits/multiFile.txt"

Driver driver

Election* election

Ballots* ballots

Tests:

Test to run that STV works with a regular run with multiple files inputed.

Compares output to expected results via integer comparison, along with testing existing audit file.

Output:

STV* STVelection

vector<Candidate> winners

vector<Candidate> losers

int seatNum

Passed or Failed: Passed

Date: 0503

The PBI, the Task Description (from Sprint Log) with Unique Testing Number:

PBI: Input multiple files

Task Description: System Testing

Unique Testing Number: 57

Team Member(s) Responsible: Manan Chaturvedi

Inputs:

```
vector<string> file_names = {"../testing/MV6.csv", "../testing/MV7.csv",  
"../testing/MV8.csv", "../testing/MV9.csv", "../testing/MV10.csv"}
```

Driver driver

Election* election

Ballots* ballots

Tests:

Test to run that MV works with a regular run with multiple files inputed.

Compares output to expected results via integer comparisons.

Outputs:

Municipal* MVelection

vector<Candidate> winners

vector<Candidate> losers

int seatNum

Passed or Failed: Passed

Date: 0503

Project Name: The project #, name of your system, and the team#

Test Stage: Indicate whether it is a unit test or a system test.

Test Date: The date the test was performed.

Test Case ID#: A unique ID is required. Decide on a naming convention and use numbering. Example: Ballot_Shuffle_1

Name(s) of Testers: List the names of anyone involved in running this test case.

Test Description: Describe briefly the test objective.

Automated: Indicate if the test is completely automated or being checked manually. (If you have methods running the tests and checking results, select “yes”. If you are manually checking results, indicate manual by selecting the “no.”)

Results: Indicate if the test passed or failed.

Step #: You will be listing the test steps in order. This number is the step number in the process.

Test Step Description: Details of the test step.

Test Data: What the test data will be for this step. Be clear on what the input data will be. If using a specific file, be clear on the name.

Expected Result: What result are you expecting from the program component or system.

Actual Result: What result were returned based on the test.

Post condition for Test: What will be true after the test has been run? Has the state of the system changed in any way?

Notes: Comments and notes for you and your team members.